Ψηφιακή Επεξεργασία Εικόνας

Εργασία 1

Παραλλαγές αλγορίθμων εξισορρόπησης και αντιστοίχισης ιστογράμματος

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών ΑΠΘ

8ο εξάμηνο

Θεσσαλονίκη, Απρίλιος 2025

Υπεύθυνη εργασίας: Μανωλίδου Ανατολή

Email: amanolid@ece.auth.gr

ΑΕΜ: 10874

Περιεχόμενα

1 calculate_hist_of_img……..……3

2 apply_hist_modification_transform……4

3 perform_hist_modification……5

3.1 Greedy Mode…….……6

3.2 Non-Greedy Mode……7

3.3 Post-Disturbance Mode……...8

4 perform_hist_eq……9

5 perform_hist_matching……..10

6 Demo.py……10

7 Τελικά Αποτελέσματα και Συμπεράσματα……11

7.1 Histogram Matching……11

7.2 Histogram Equalization……..13

1 calculate_hist_of_img

Παρακάτω παρατίθεται το κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **calculate_hist_of_img:**

```
def calculate_hist_of_img(
    img_array: np.ndarray,
    return_normalized: bool
) -> Dict:

    #Checking if the given image is grayscale or not
    if len(img_array.shape) == 2:  #Grayscale Image

        unique, counts = np.unique(img_array, return_counts=True) #Counting
occurrences of each element in img_array

        if return_normalized:
            counts = counts / img_array.size

        hist = {float(u): float(c) for u, c in zip(unique, counts)}
        return hist

    elif len(img_array.shape) == 3 and img_array.shape[2] == 3: #RGB Image

        print("This is not a grayscale image. Exiting...")
        return None
```

Αρχικά έχουν οριστεί οι είσοδοι και οι έξοδοι της συνάρτησης όπως αυτά δόθηκαν στην εκφώνηση της άσκησης. Στη συνέχεια, ελέγχουμε τις διαστάσεις του πίνακα *img_array*, σε περίπτωση που έχουμε τρεις διαστάσεις, εκ των οποίων η μια έχει μέγεθος ίσο με 3, τότε πρόκειται για πίνακα που προήλθε από RGB εικόνα οπότε εκτυπώνεται ένα μήνυμα λάθους και επιστρέφουμε *None*. Στην περίπτωση όμως που έχουμε πίνακα *img_array* δυο διαστάσεων τότε πρόκειται για grayscale εικόνα οπότε προχωράμε στη παραγωγή του ιστογράμματος. Πιο συγκεκριμένα:

1. Μέσω της συνάρτησης *np.unique*, αποσπάμε από τον πίνακα μας όλα τα ξεχωριστά values, δηλαδή τις στάθμες, και την απόλυτη συχνότητα εμφάνισης που τους αντιστοιχεί
2. Σε περίπτωση που έχουμε επιλέξει κανονικοποίηση *(return_normalized == True),* διαιρούμε όλες τις συχνότητες με το μέγεθος του πίνακα

3. Δημιουργούμε το ζητούμενο dictionary της εξόδου και συγκεκριμένα ως *keys* εκχωρούμε τις στάθμες τις εικόνας *(unique)* ενώ ως *values* την αντίστοιχη συχνότητα εμφάνισης *(counts)*

2 apply_hist_modification_transform

Παρακάτω παρατίθεται το κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **apply_hist_modification_transform:**

```
def apply_hist_modification_transform(
    img_array: np.ndarray,
    modification_transform: Dict
) -> np.ndarray:

    modified_img = np.zeros_like(img_array, dtype=float) #Creating an empty array
of the same shape as the input image

    #Iterating through each pixel in the image and applying the transformation
    for i in range(img_array.shape[0]):
        for j in range(img_array.shape[1]):
            key = img_array[i, j]
            mapped_val = modification_transform.get(key, 0.0)
            modified_img[i, j] = mapped_val

    return modified_img
```

Αρχικά έχουν οριστεί οι είσοδοι και οι έξοδοι της συνάρτησης όπως αυτά δόθηκαν στην εκφώνηση της άσκησης. Στη συνέχεια:

1. Δημιουργούμε τον πίνακα εξόδου *modified_img*, ως ένα μηδενικό πίνακα με διαστάσεις ίδιες με αυτές του πίνακα εισόδου και στοιχεία τύπου *float*
2. Με ένα διπλό for loop ανατρέχουμε από όλα τα στοιχεία του πίνακα εισόδου και πιο συγκεκριμένα, αναθέτουμε στη μεταβλητή key το τρέχων στοιχείο του πίνακα ενώ στη *mapped_val* το *value* του *modification_transform* dictionary για το τρέχων *key*, δηλαδή τη στάθμη εξόδου για την αντίστοιχη στάθμη εισόδου. Σε περίπτωση που δεν υπάρχει αντιστοίχιση η *mapped_val* παίρνει τη τιμή *0*
3. Αναθέτουμε τη νέα τιμή στην εικόνα εξόδου

3 perform_hist_modification

Παρακάτω παρατίθεται το πρώτο κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **perform_hist_modification:**

```
def perform_hist_modification(
    img_array: np.ndarray,
    hist_ref: Dict,
    mode: str
) -> np.ndarray:

    input_hist = calculate_hist_of_img(img_array, return_normalized=False)
    if input_hist is None:
        print("Error: Unable to calculate histogram for the image.")
        return None

    input_levels = list(sorted(input_hist.keys())) #Sorting the keys of the input
histogram
    input_counts = np.array([input_hist[key] for key in input_levels]) #Creating
an array of counts for the input histogram
    total_samples = img_array.size

    output_levels = np.linspace(0.0, 1.0, len(hist_ref)) #Creating an array of
output levels
    output_probs = np.array(list(hist_ref.values())) #Creating an array of the
probabilities
    output_counts = output_probs * total_samples #(N · pref(gi))

    greedy = {}
    non_greedy = {}
    post_disturbance = {}
```

Αρχικά έχουν οριστεί οι είσοδοι και οι έξοδοι της συνάρτησης όπως αυτά δόθηκαν στην εκφώνηση της άσκησης. Έπειτα:

1. Ορίζουμε ως *input_hist* το dictionary της εικόνας εισόδου (χρησιμοποιώντας τη προηγούμενη συνάρτηση μας **calculate_hist_of_img),** σε περίπτωση που η συνάρτηση επιστρέψει *None* σημαίνει πως υπάρχει κάποιο θέμα με τις διαστάσεις του *img_array* και έτσι εκτυπώνουμε μήνυμα λάθους και επιστρέφουμε *None*.
2. Έπειτα, από το *input_hist* dictionary εξάγουμε τα *keys*, δηλαδή τις στάθμες εισόδου, και τις αναθέτουμε στη μεταβλητή *input_levels* ενώ δημιουργούμε και τον πίνακα *input_counts*, που περιέχει τις συχνότητες εμφάνισης για κάθε στάθμη εισόδου.

3. Δημιουργούμε έναν πίνακα από επίπεδα εξόδου τα οποία είναι ομοιόμορφα κατανεμημένα μεταξύ *0.0* και *1.0* και το πλήθος τους ισούται με το πλήθος της εικόνας αναφοράς
4. Υπολογίζουμε τις πιθανότητες από τα στοιχεία του πίνακα της εικόνας αναφοράς μας. Όσο αναφορά τον υπολογισμό των πιθανοτήτων, θεωρούμε πως το dictionary της εικόνας αναφοράς προήλθε από τη συνάρτηση **calculate_hist_of_img** με τη μεταβλητή *return_normalized == True*, οπότε οι πιθανότητες ισούνται με τα *values* του dictionary, χωρίς να χρειαστεί η διαίρεση με το 255.
5. Υπολογίζουμε το κατώφλι όπως αυτό μας δόθηκε *(N · pref(gi))*
6. Τέλος, δημιουργούμε τρία dictionaries, ένα για το κάθε mode.

Σε περίπτωση που δεν έχει δοθεί κάποια από τις επιτρεπτές τιμές για τη μεταβλητή *mode,* επιστρέφουμε μήνυμα λάθους και επιστρέφουμε *None,* όπως φαίνεται παρακάτω:

```
if mode not in ['greedy', 'non-greedy', 'post_disturbance']:
        print("Invalid mode selected. Exiting...")
        return None
```

3.1 Greedy Mode

Παρακάτω παρατίθεται το πρώτο κομμάτι του κώδικα στο υλοποιείται το **Greedy** mode**:**

```
  if mode == "greedy":

        j = 0 #Index for output levels
        count_g = 0 #Count of input levels

        #Iterating through the input levels and mapping them to output levels
        for i in range(len(input_levels)):

            count_g += input_counts[i]
            greedy[input_levels[i]] = output_levels[j]

            if count_g >= output_counts[j] and count_g - input_counts[i] <
output_counts[j]:
                j += 1
                count_g = 0

        return apply_hist_modification_transform(img_array, greedy)
```

Αρχικά, ορίζουμε τις παρακάτω μεταβλητές και τις θέτουμε ίσες με το μηδέν

- count_g: πόσα samples έχουν ανατεθεί στη τρέχουσα στάθμη της εικόνας εξόδου
- j: το index της τρέχουσας στάθμης της εικόνας εξόδου

Έπειτα, με ένα for loop ανατρέχουμε από όλες τις στάθμες της εικόνας εισόδου και:

1. Αναθέτουμε στη μεταβλητή *count_g* τη τρέχουσα στάθμη εισόδου.
2. Στο *greedy* dictionary μας, αντιστοιχούμε τη στάθμη εισόδου με τη στάθμη εξόδου
3. Στη συνέχεια ελέγχουμε αν ικανοποιείται η συνθήκη *count_g >= img_array.size * prob_g[j]*, ώστε να προχωρήσουμε στην επόμενη στάθμη εξόδου. Βέβαια αυτή η συνθήκη μόνη της δεν αρκεί, οπότε ελέγχουμε και τη συνθήκη *count_g - values[i] < img_array.size * prob_g[j]* η οποία μας διαβεβαιώνει πως η τρέχουσα στάθμη εισόδου είναι αυτή η οποία «ξεπέρασε» το όριο *img_array.size * prob_g[j]*.
4. Σε περίπτωση που ικανοποιούνται όλες οι προαναφερθείσες συνθήκες, αυξάνουμε το δείκτη της στάθμης εξόδου *j* και μηδενίζουμε τη μεταβλητή *count_g*.
5. Τέλος, χρησιμοποιώντας τη συνάρτηση **apply_hist_modification_transform,** εφαρμόζουμε στην εικόνα μας το *greedy* μετασχηματισμό που μόλις φτιάξαμε

3.2 Non-Greedy Mode

Παρακάτω παρατίθεται το πρώτο κομμάτι του κώδικα στο υλοποιείται το **Non-Greedy** mode**:**

```
    if mode == "non-greedy":

        j = 0 #Index for output levels
        count_g = 0 # Count of input levels

        for i in range(len(input_levels)):

            count_g += input_counts[i]
            deficiency = output_counts[j] - count_g

            if deficiency >= input_counts[i] / 2: #Mapping to the current output
level

                non_greedy[input_levels[i]] = output_levels[j]
            else: #Moving on to the next output level

                j += 1
                non_greedy[input_levels[i]] = output_levels[j]
                count_g = input_counts[i] #Reset count_g to the current input
level count

        return apply_hist_modification_transform(img_array, non_greedy)
```

Με την ίδια λογική όπως και στο ‘’Greedy’’ mode θέτουμε τις μεταβλητές *count_g*, και *j*. Στη συνέχεια:

1. Με ένα for loop ανατρέχουμε από όλες τις στάθμες της εικόνας εισόδου
2. Αναθέτουμε στη μεταβλητή *count_g* τη τρέχουσα στάθμη εισόδου
3. Υπολογίζουμε τη μεταβλητή *deficiency* ως *deficiency = (img_array.size * prob_g[j]) - count_g*.
4. Σύμφωνα και με τις απαιτήσεις της εργασίας, σε περίπτωση που η μεταβλητή *deficiency* είναι ίση ή μεγαλύτερη από το όριο *values[i])/2*, τότε παραμένουμε στο ίδιο επίπεδο εξόδου και έτσι αντιστοιχίζουμε το *input_levels[i]* σε αυτό το επίπεδο
5. Σε περίπτωση όμως που η μεταβλητή *deficiency* δε ξεπεράσει το όριο *values[i])/2,* πρέπει να προχωρήσουμε στο επόμενο επίπεδο εξόδου, γι’ αυτό και αυξάνουμε το δείκτη *j,* αντιστοιχίζουμε το *input_levels[i]* στο νέο επίπεδο και τέλος επαναφέρουμε το *count_g* ώστε να ισούται μόνο με τη τρέχουσα στάθμη εισόδου.
6. Τέλος, χρησιμοποιώντας τη συνάρτηση **apply_hist_modification_transform,** εφαρμόζουμε στην εικόνα μας το *non-greedy* μετασχηματισμό που μόλις φτιάξαμε

3.3 Post-Disturbance Mode

Παρακάτω παρατίθεται το πρώτο κομμάτι του κώδικα στο υλοποιείται το **Post-Disturbance** mode**:**

```
if mode == "post-disturbance":

        d = 1.0 / (len(input_levels) - 1) #Calculating the distance between
levels

        noise = np.random.uniform(-d/2, d/2, size=img_array.shape) #Generating
random uniform noise

        disturbed_img = img_array + noise #Adding noise to the image
        disturbed_img = np.clip(disturbed_img, 0.0, 1.0) #Clipping the values to
be between 0 and 1

        post_disturbance = perform_hist_modification(disturbed_img, hist_ref,
mode="greedy") #Performing greedy histogram modification on the disturbed image
        return post_disturbance
```

Όσο αφορά το Post-Disturbance mode:

1. Υπολογίζουμε την απόσταση μεταξύ των στάθμεων της εικόνας εισόδου
2. Δημιουργούμε τυχαίο θόρυβο από ομοιόμορφη κατανομή στο διάστημα *[-d/2,d/2]* και τον προσθέτουμε στην εικόνα εισόδου μας.

3. Χρησιμοποιούμε τη συνάρτηση *np.clip* ώστε να βεβαιωθούμε πως οι νέες τιμές των pixels δεν θα ξεπερνάνε το εύρος *[0.0,1.0]*
4. Στη *disturbed* εικόνα εφαρμόζουμε το *greedy* μετασχηματισμό

4 perform_hist_eq

Παρακάτω παρατίθεται το πρώτο κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **perform_hist_eq:**

```
def perform_hist_eq(
    img_array: np.ndarray,
    mode: str
) -> np.ndarray:

    unique_vals = np.unique(img_array) #Finding unique values in the image array
    L = len(unique_vals)
    uniform_hist = {v: 1.0/L for v in unique_vals} #Creating a uniform histogram

    return perform_hist_modification(img_array, uniform_hist, mode)
```

Ζητούμενο της συγκεκριμένης συνάρτησης ήταν να επιτύχουμε την εξισορρόπηση της εικόνας εισόδου *img_array*, χρησιμοποιώντας τη συνάρτηση **perform_hist_modification.** Όπως αναφέρεται και στην εκφώνηση της εργασίας, σκοπός της εξισορρόπησης είναι να παραχθεί μια εικόνα της οποίας το ιστόγραμμα έχει στάθμες οι οποίες είναι όσο το δυνατό πιο ομοιόμορφα κατανεμημένες γίνεται. Με βάση όλα τα παραπάνω έχουμε ως εξής:

1. Μέσω της συνάρτησης *np.unique*, αποσπάμε από τον πίνακα μας όλα τα ξεχωριστά values, δηλαδή τις στάθμες
2. Αποθηκεύουμε στη μεταβλητή *L* το πλήθος αυτών των στάθμεων
3. Σε κάθε μοναδική στάθμη δίνουμε ίση πιθανότητα εμφάνισης *(1/L)* και δημιουργούμε ένα dictionary αντιστοίχισης αρχικών στάθμεων με τι νέες ίσο πιθανές
4. Χρησιμοποιούμε τη συνάρτηση **perform_hist_modification,** όπου ως εικόνα αναφοράς θεωρούμε τη *uniform_hist*

5 perform_hist_matching

Παρακάτω παρατίθεται το πρώτο κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **perform_hist_matching:**

```
def perform_hist_matching(
    img_array: np.ndarray,
    img_array_ref: np.ndarray,
    mode: str
) -> np.ndarray:

    hist_ref = calculate_hist_of_img(img_array_ref, return_normalized=True)

    return perform_hist_modification(img_array, hist_ref, mode)
```

Σε αυτή τη συνάρτηση απλά δημιουργούμε το ιστόγραμμα της εικόνας αναφοράς και έπειτα καλούμε τη συνάρτηση **perform_hist_modification**

6 Demo.py

Όσο αφορά το demo.py, αρχικά δημιουργούμε τη παρακάτω συνάρτηση, η οποία παράγει ένα vertical bar για αναπαράσταση του grayscale spectrum.

```
def draw_grayscale_bar(ax): #Create a vertical grayscale bar
    gradient = np.linspace(0, 1, 256).reshape(256, 1)
    ax.imshow(gradient, cmap="gray", aspect="auto")
    ax.set_yticks([0, 64, 128, 192, 255])
    ax.set_yticklabels(["0.0", "0.25", "0.5", "0.75", "1.0"])
    ax.set_xticks([])
    ax.set_box_aspect(6)
```

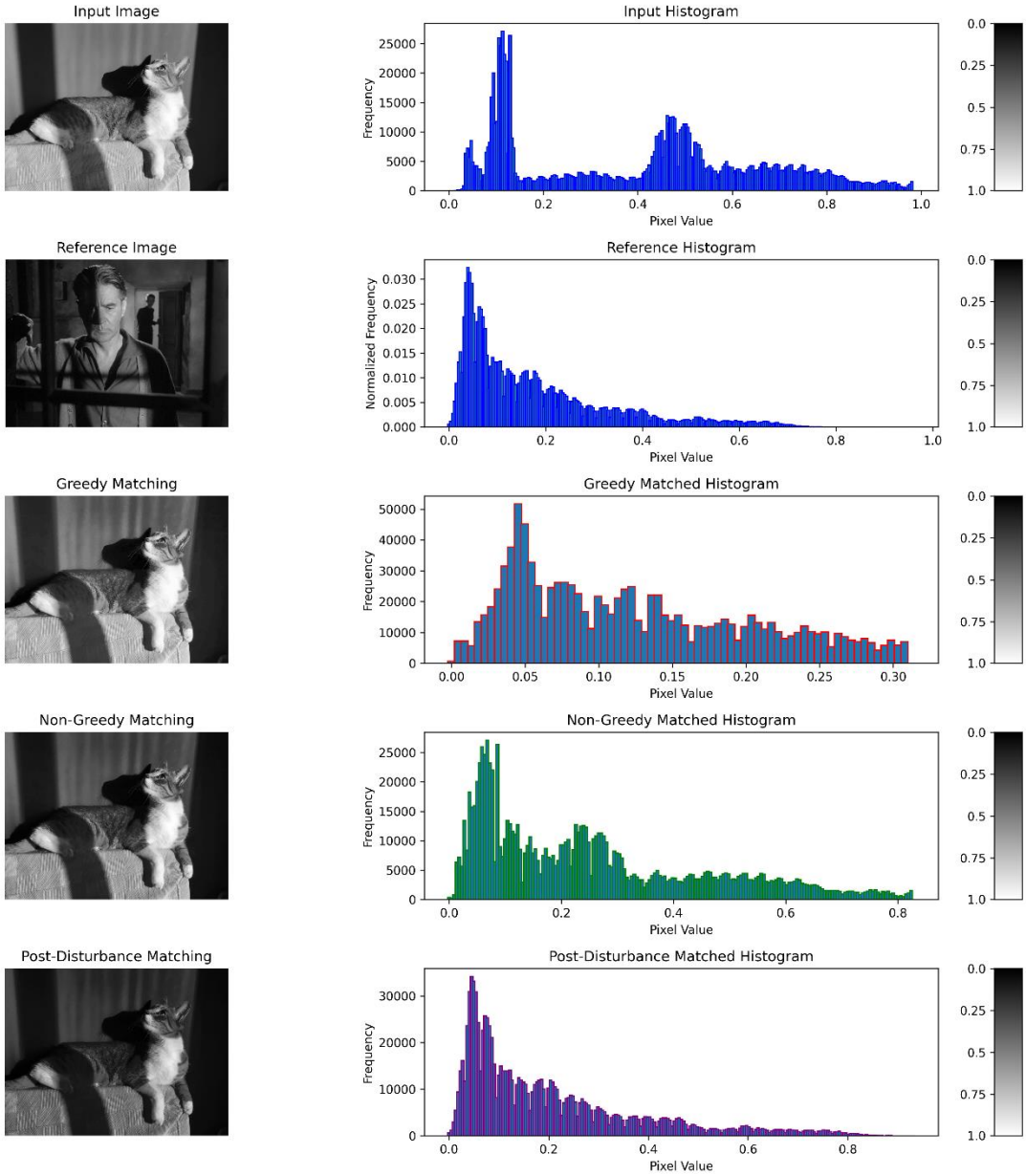
Λόγω μεγάλης έκτασης, το υπόλοιπο κομμάτι του κώδικα δε παρατίθεται στην αναφορά, παρόλο αυτά παρακάτω συνεχίζουμε με μία μικρή περιγραφή του.

Στη συνέχεια, μέσω των κατάλληλων paths φορτώνουμε τις εικόνες εισόδου και αναφοράς, τις μετατρέπουμε σε ασπρόμαυρες και τις κανονικοποιούμε. Έπειτα υπολογίζουμε το ιστόγραμμα αυτών των εικόνων μέσω της συνάρτησης **calculate_hist_of_img**, εφαρμόζουμε το matching και για τα τρία modes μέσω της συνάρτησης **perform_hist_modification** και τέλος υπολογίζουμε το ιστόγραμμα των εικόνων εξόδων του matching μέσω της συνάρτησης **calculate_hist_of_img.** Τέλος, φτιάχνουμε τα plots κατάλληλα. Η ίδια διαδικασία γίνεται και για τη περίπτωση της εξισορρόπησης του ιστογράμματος της εικόνας εισόδου με τη μόνη διαφορά πως αντί της **perform_hist_modification** χρησιμοποιούμε τη συνάρτηση **perform_hist_eq.**

7 Τελικά Αποτελέσματα και Συμπεράσματα

7.1 Histogram Matching

Παρακάτω, παρατίθεται το plot έπειτα από την εφαρμογή του histogram matching:



Όσο αφορά την εικόνα εισόδου, παρατηρούμε πως εμφανίζει περιοχές με αυξημένη συχνότητα εμφάνισης γεγονός που δικαιολογεί την «ανομοιόμορφη» φωτεινότητα της. Ειδικά, οι δύο λωρίδες σκιάς που βλέπουμε, αυξάνουνε αρκετά τη συχνότητα των pixels με τιμή κοντά στο 0, γι΄αυτό και βλέπουμε αυτό το spike στις τιμές κοντά στο 0.

Όσο αφορά την εικόνα αναφοράς, προφανώς πρόκειται για μια εικόνα αρκετά σκοτεινή, με τη πλειοψηφία των pixels της να έχουν τιμές στο εύρος *(0,0.17),* ελάχιστα pixels στις ενδιάμεσες «γκρίζες» αποχρώσεις και καθόλου pixels κοντά στο 1, δηλαδή κοντά στο χρώμα λευκό.

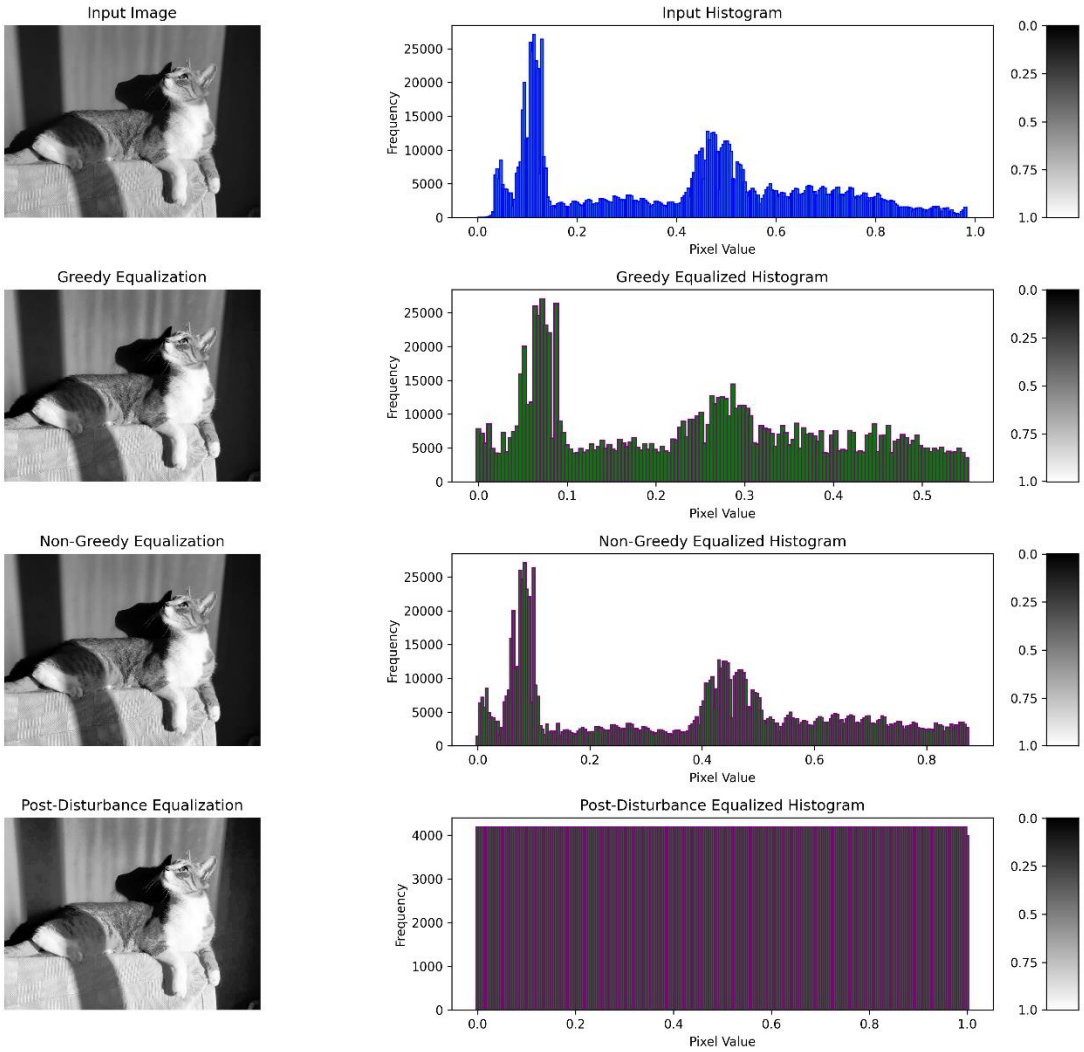
Προχωρώντας στο matching, αρχικά θα ασχοληθούμε το Greedy mode. Με μια πρώτη ματιά, βλέπουμε πως η εικόνα μας έχει «σκουραίνει» αρκετά και το ιστόγραμμα της πλέον ακολουθεί τη κατανομή του ιστογράμματος αναφοράς. Πιο συγκεκριμένα, παρατηρούμε μια μείωση στο πλήθος των στάθμεων, γεγονός που δικαιολογείται καθώς η συγκεκριμένη μέθοδος αντιστοιχεί όσες περισσότερες στάθμες εισόδου σε στάθμες εξόδου.

Στο Non-Greedy mode και πάλι βλέπουμε πως η εικόνα μας έχει «σκουραίνει» αρκετά και το ιστόγραμμα της πλέον ακολουθεί τη κατανομή του ιστογράμματος αναφοράς. Αυτή τη φορά όμως βλέπουμε μια καλύτερη προσέγγιση ως προς το ιστόγραμμα αναφοράς καθώς σε αυτή τη μέθοδο έχουμε ορίσει καλύτερα το κατώφλι για τη αντιστοίχιση σε στάθμες εξόδου.

Τέλος, στο Post-Disturbance mode παρατηρούμε τα καλύτερα αποτελέσματα καθώς το ιστόγραμμα εξόδου προσεγγίζει σχεδόν πανομοιότυπα αυτό της εικόνας αναφοράς.

7.2 Histogram Equalization

Παρακάτω, παρατίθεται το plot έπειτα από την εφαρμογή του histogram equalization:



Στο greedy mode, παρατηρούμε μια αύξηση στις συχνότητες που δεν εμφάνιζαν spikes στο αρχικό μας ιστόγραμμα. Ως αποτέλεσμα έχουμε μια πιο ομοιόμορφη κατανομή που προσεγγίζει και το ζητούμενο της εξισορρόπησης.

Από την άλλη, στο non-greedy mode δε παρατηρούμε ιδιαίτερη αλλαγή παρά μόνο στο αρχικό εύρος *(0.8,1)* το οποίο φαίνεται να έχει πιο ομοιόμορφη κατανομή.

Τέλος στο Post-Disturbance mode παρατηρούμε πως το ιστόγραμμα της εξόδου ακολουθεί πλήρως τη κατανομή από το ιστόγραμμα αναφοράς. Εδώ να παραθέσω πως το συγκεκριμένο αποτέλεσμα μου φαίνεται αρκετά περίεργο και δε μπορώ να ερμηνεύσω το λόγο που έχουμε ένα τελείως ομοιόμορφο αποτέλεσμα. Παρόλο αυτά, η εικόνα εξόδου φαίνεται να έχει χαμηλωμένες τις εντάσεις όπου αντιστοιχούν στις αρχικές περιοχές με τα spikes, γεγονός που συμβαδίζει με το ιστόγραμμα εξόδου.